

Iterable Interface

Page No.:

YOUVA
PELLAR

Date:

Collection

- ArrayList
- LinkedList
- Stack / Queue
- Set / Map

```
List<Integer> list = new ArrayList<>();
```

2	3	5	9	7	8
0	1	2	3	4	5

 = (6)

```
for(int i=0; i< list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

Data Structures

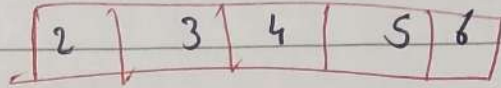
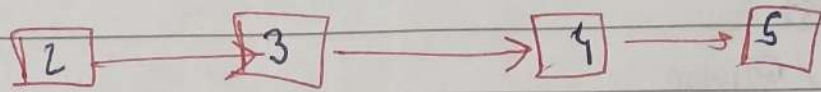
ordering
maintain

not maintain
ordering

- ArrayList
- LinkedList
- & any type
of List

- Set
- Map
- Stack
- Queue

List<Integer> list = new LinkedList();



```
for (i=0; i < list.size(); i++) {  
    sout(list.get(i));  
}
```

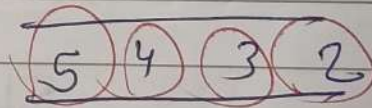
→ $O(N^2)$

Collection → Iterate

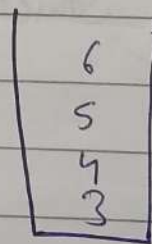
↳ all are not indexed

↳ Linked list → get(i) → unoptimized

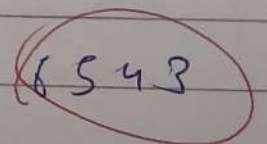
Queue :



Stack :



LIFO

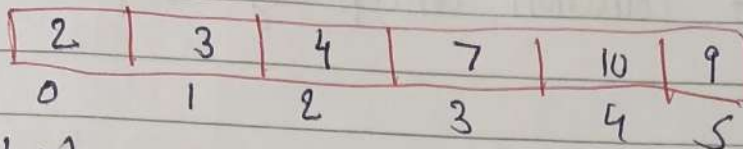


Solution: Iterable Interface

* Iterable

— it is the root interface that represent any object whose element can be traversed one by one

ArrayList:



Linked List

Set

Queue



X

hasNext()

next()

Iterable

iterator()

Iterator

hasNext()
next()

interface Iterable {

Iterator ~~iterator~~ iterator();

}

interface Iterator {

hasNext();

next();

}

Iterator

ArrayList Iterator

ArrayList

Iterator → iterator() {

return new
ArrayListIterator();

}

class ArrayListIterator

implements Iterator {

hasNext() {

next() {

}


```

List<> list = new ArrayList<>();
Iterator it = list.iterator();
it.hasNext();
it.next();

```

* How Different Collection conceptually implement Iterable / Iterator

ArrayList

```

class ArrayList implements Iterable {
    private Integer[] arrs;
    int private int size;

```

@Override

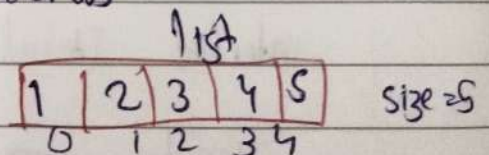
```

    public Iterator iterator() {
        return new ArrayListIterator();
    }

```

}

inner class



```

class ArrayListIterator implements Iterator { int pos = 0;

```

@Override

```

    public boolean hasNext() {
        return (pos < size);
    }

```

@Override

```

    public Integer next() {
        int x = arr[pos];

```

pos++;

return x;

→ return arr[pos++];

}

```
main() {
    Iterator it = list.iterator();
```

```
    while (it.hasNext()) {
        sout(it.next());
    }
```

Now LinkedList:

```
class LinkedList implements Iterable {
    static class Node {
```

```
        int data;
        Node next;
```

```
    }
```

```
    Node head;
```

```
    @Override
```

```
    Iterator iterator() {
```

```
        return new LinkedListIterator();
    }
```

```
class LinkedListIterator implements Iterator { Node current = head;
```

```
    @Override
```

```
    boolean hasNext() {
```

```
        return current != null;
```

```
    }
```

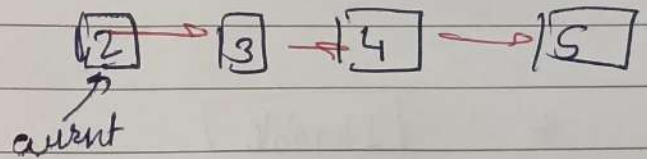
```
    @Override
```

```
    Integer next() {
```

```
        int data = current.data;
```

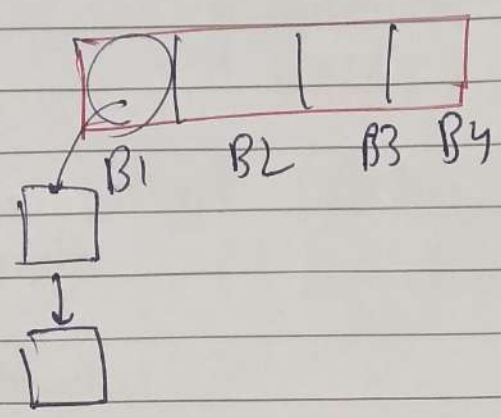
```
        current = current.next;
```

```
        return data;
    }
```



```
    while (it.hasNext()) {
        sout(it.next());
    }
```


Set / Map
 ↳ Hashed



Tree set / Tree Map

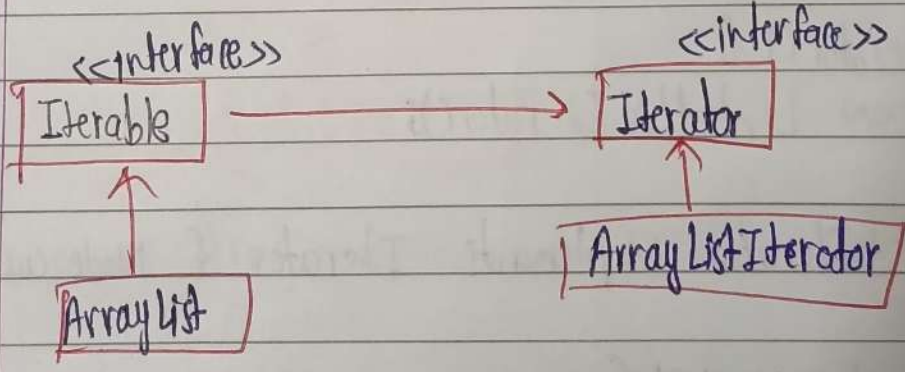
BST
 ↳ inorder()

Queue / Stack
 ↳ ArrayList

*

Iterable
 ↳ iterator()
 ↳ for Each()
 ↳ split Iterator()

Iterator
 ↳ hasNext()
 ↳ next()
 ↳ remove()
 ↳ For Each Remaining()
 functional interface



class ArrayList implements Iterator {
 hasNext()
 next()
 }

problems: ① Separation of concern
②

✓ Problems

class MyArrayList {

int[] ^{arr} ~~pos~~ = {10, 20, 30};

int size = 3;

private int pos = 0; void reset() {
pos = 0;

public boolean hasNext() {
return pos < size;

}
public boolean next() {
return arr[pos++];
}

}

10 20 30

MyArrayList l = new MyArrayList();
while (l.hasNext()) {
 sout(l.next());
}

l.reset();
while (l.hasNext()) {
 sout(l.next());
}

l = new MyArrayList();
l.reset();

a = 10

b = 20

10, 20
10, 30

No result now

while (l.hasNext()) {
 int a = l.next();
 while (l.hasNext()) {
 int b = l.next();
 sout(a, b);
 }
}